RedM Command Center V3

Complete System Walkthrough

A comprehensive technical inspection of the AI-powered anti-human trafficking social media campaign generation system. Every source file has been read and verified line-by-line.

Project	Fully_functional_MVP_V3
Architecture	FastAPI + LangGraph + SQLite
LLM Models	GPT-4o · GPT-4o-mini · Gemini 3.1 Flash
Date	April 19, 2026
Classification	Internal — Confidential

Table of Contents

1 Executive Summary

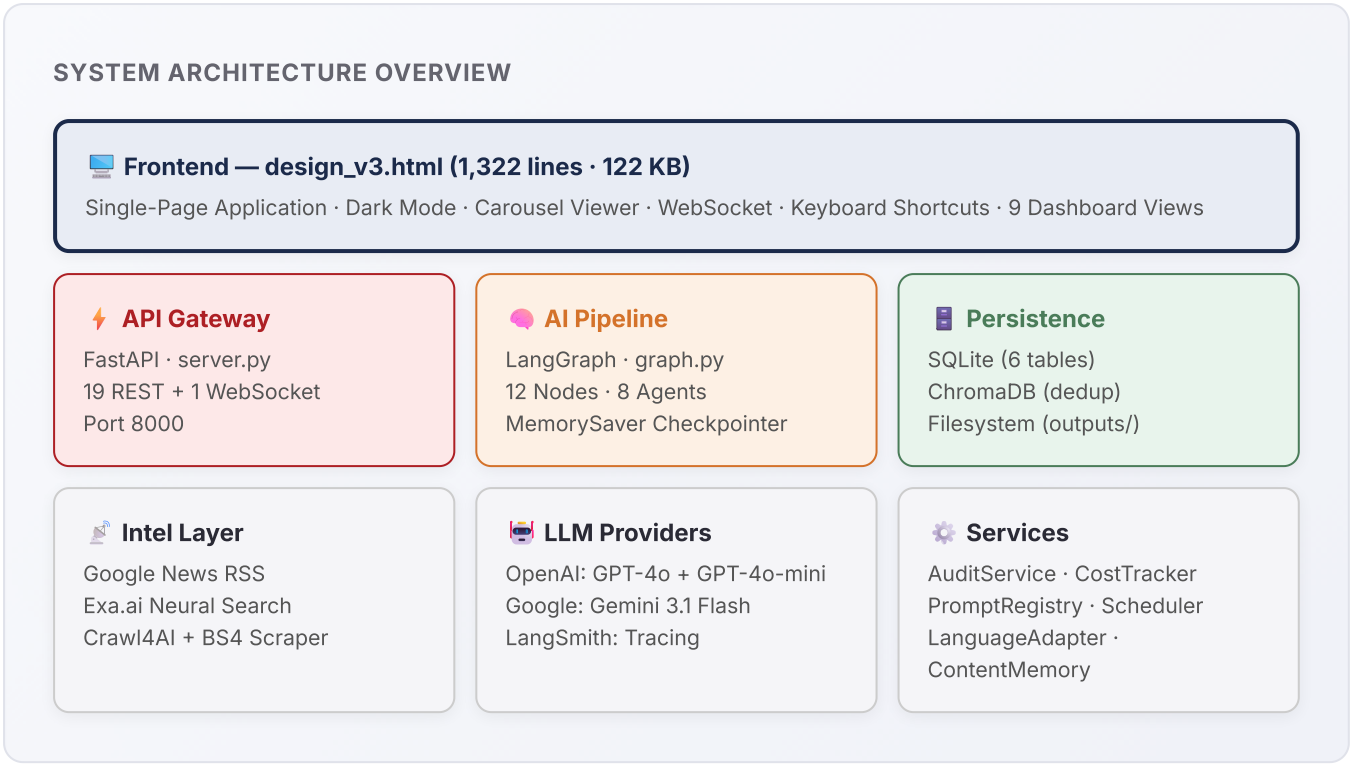
RedM Command Center V3 is an AI-powered social media campaign generator designed for anti-human trafficking awareness organizations. Given a single news story about human trafficking, the system automatically produces a complete **5-post weekly campaign** (Monday through Friday), where each post has a unique content type, an AI-generated caption with 2 alternative variants, AI-generated imagery, multi-platform renders for 5 social media platforms, quality scoring, and ethical compliance checks — all before human review.

The system is built as a full-stack application with a **FastAPI backend** (19 REST endpoints + WebSocket), a **LangGraph-based AI pipeline** (12 processing nodes), **SQLite persistence** (6 database tables), and a polished **single-page HTML dashboard** with dark mode, carousel post viewer, keyboard shortcuts, and real-time WebSocket updates.

Key Capabilities

- **Intelligence Scanning:** Automated Google News RSS scanning across 12 trusted news domains
- **Story Bypass Architecture:** Pre-selected stories skip the LLM trend analyzer, saving time and API costs
- **5-Post Campaign Generation:** Reactive (Mon), Stat Card (Tue), Explainer (Wed), Action (Thu), Hope (Fri)
- **Multi-Model AI Pipeline:** GPT-4o for planning, GPT-4o-mini for writing/quality/compliance, Gemini 3.1 Flash for images
- **Ethical Compliance Engine:** YAML-driven rules with regex + LLM validation and auto-fix
- **Multi-Platform Rendering:** Images rendered to IG Square, IG Stories, X Card, LinkedIn, Facebook
- **Human-in-the-Loop Review:** Pipeline pauses for batch approval before publishing
- **Full Audit Trail:** Every action logged with timestamps for accountability
- **Multi-Language:** Translation to Spanish, French, Portuguese with cultural adaptation
- **Cost Tracking:** Per-token API cost logging across all LLM calls

2 System Architecture



Layer Descriptions

Frontend (design_v3.html): A single-page HTML/CSS/JS application that communicates with the backend via fetch() API calls and WebSocket connections. Uses Inter and JetBrains Mono fonts, CSS custom properties for light/dark theming, and vanilla JavaScript with no framework dependencies.

API Gateway (server.py): A FastAPI application (566 lines) running on Uvicorn at port 8000. CORS is open for local development. Campaign generation is delegated to background threads via `asyncio.to_thread`. Static file serving for generated images at `/outputs/`.

AI Pipeline (graph.py): A LangGraph StateGraph compiled with MemorySaver checkpointer and `interrupt_after=['batch_review']` for human-in-the-loop control. Uses conditional edges for story bypass routing and the 5-post generation loop.

Persistence: SQLite via SQLModel (6 tables) for structured data. ChromaDB for semantic dedup (0.85 cosine threshold). Filesystem for generated images and ZIP bundles. YAML configs for content rules and audience profiles.

3 Complete File Inventory

Root Level (3 HTML Files)

FILE	LINES	SIZE	PURPOSE
`design_v3.html`	1,322	122 KB	Primary UI — full SPA dashboard with 9 views
`design_complete.html`	~1,218	91 KB	Earlier/simpler UI version
`architecture_diagram.html`	~515	23 KB	Interactive architecture visualization

Backend — Top Level

FILE	LINES	SIZE	PURPOSE
`server.py`	566	22 KB	FastAPI server — 19 endpoints + WebSocket, serves dashboard at /
`config.py`	65	2 KB	Global config: 13 news domains, 5 platform specs, 4 languages
`requirements.txt`	35	491 B	18 pinned Python dependencies
`.env`	9	555 B	API keys: OpenAI, Gemini, Exa, LangSmith
`redm_v3.db`	—	~17 MB	SQLite database with real campaign data from 5 runs

Backend — Core Data (src/)

FILE	LINES	SIZE	PURPOSE
`models.py`	122	5 KB	6 SQLModel table definitions: Campaign, Post, IntelStory, AuditEntry, PromptVersion, CostLog
`state.py`	81	4 KB	LangGraph AgentState TypedDict + CampaignPost Pydantic model with 16 fields

Backend — AI Agents (src/agents/) — 8 Files

FILE	LINES	LLM MODEL	PURPOSE
`trend_analyzer.py`	126	`gpt-4o-mini`	Scans news → identifies single most important trafficking story
`audience_analyzer.py`	100	`gpt-4o-mini`	Matches story to 1 of 6 audience profiles → writing + visual briefs
`campaign_planner.py`	136	`gpt-4o`	Transforms 1 story into 5 structured CampaignPost briefs

FILE	LINES	LLM MODEL	PURPOSE
`writer.py`	175	`gpt-4o-mini`	Generates caption + 2 variants per post; per-type system prompts
`quality_gate.py`	103	`gpt-4o-mini`	LLM-as-Judge: scores 0-10 across hallucination, tone, CTA, source
`compliance_checker.py`	140	`gpt-4o-mini`	Regex quick-check + LLM deep-check against YAML rules; auto-fixes
`image_generator.py`	321	`gpt-4o-mini` + `gemini-3.1-flash`	GPT builds prompt → Gemini generates image → Pillow composites text
`publisher.py`	91	`gpt-4o-mini`	Formats captions for X/IG; generates HTML preview (NOT in pipeline)

Backend — Services (src/services/) — 7 Files

FILE	LINES	LLM?	PURPOSE
`intel_service.py`	122	Declared unused	News scan → persist to IntelStory table; ranker built but never called
`content_memory.py`	102	No	ChromaDB semantic dedup (0.85 cosine threshold); built but not wired
`cost_tracker.py`	91	No	Per-call token usage + cost to CostLog table; pricing for all 3 models
`audit_service.py`	92	No	Immutable action log → AuditEntry table
`prompt_registry.py`	101	No	Prompt versioning + running avg quality scores for A/B testing
`language_adapter.py`	92	`gpt-4o-mini`	Translates campaigns to ES/FR/PT with cultural adaptation
`scheduler.py`	162	No	APScheduler timed publishing + ZIP bundle creator with manifest.json

Backend — Tools (src/tools/) — 5 Files

FILE	LINES	PURPOSE
`google_news.py`	93	Google News RSS parser; domain-restricted to 12 trusted sources; no API key
`exa_search.py`	85	Exa.ai neural search across 13 news domains + X.com social monitoring
`crawl4ai_scraper.py`	50	Async web scraper: Crawl4AI → BeautifulSoup4 fallback
`api_wrapper.py`	58	Retry decorator (3x, exponential backoff) + auto-scrape thin results
`platform_renderer.py`	96	Smart-resizes images to 5 platform sizes, protects bottom 25% text zone

Backend — Configuration (config/)

FILE	LINES	PURPOSE
`content_rules.yaml`	65	6 language rules, 7 content rules, content mix order, post structure
`audience_profiles.md`	65	6 audience personas with trigger keywords, tone, focus, CTA, visual style
`visual_philosophy.md`	63	"Less Is More": 1 subject, 50-60% negative space, 2-3 tones, bottom 25% reserved
`PlayfairDisplay-Bold.ttf`	—	Serif font for headline overlays on generated images
`Roboto-Bold/Regular.ttf`	—	Sans font for fact + source line overlays on generated images

4 LLM Models & Usage Matrix

The system uses exactly **3 distinct LLM models** across 2 providers. The Campaign Planner is the only agent using the full GPT-4o model (the "strategic brain"), while all other text agents use the cheaper GPT-4o-mini. Image generation is handled exclusively by Google Gemini.

> ⚠️ **CRITICAL DETAIL**
>
> The Campaign Planner uses `gpt-4o` (line 56 in campaign_planner.py) — NOT gpt-4o-mini. This is intentional: it is the strategic brain that creates the 5-post narrative arc, requiring the highest reasoning capability.

Model Overview

MODEL	PROVIDER	TEMPERATURE	USED BY
`gpt-4o`	OpenAI	0.3	Campaign Planner only (strategic planning)
`gpt-4o-mini`	OpenAI	0 / 0.2 / 0.3 / 0.7	All other text agents (7 agents, 10 instances)
`gemini-3.1-flash-image-preview`	Google	N/A	Image Generator only

Detailed Agent → Model Mapping

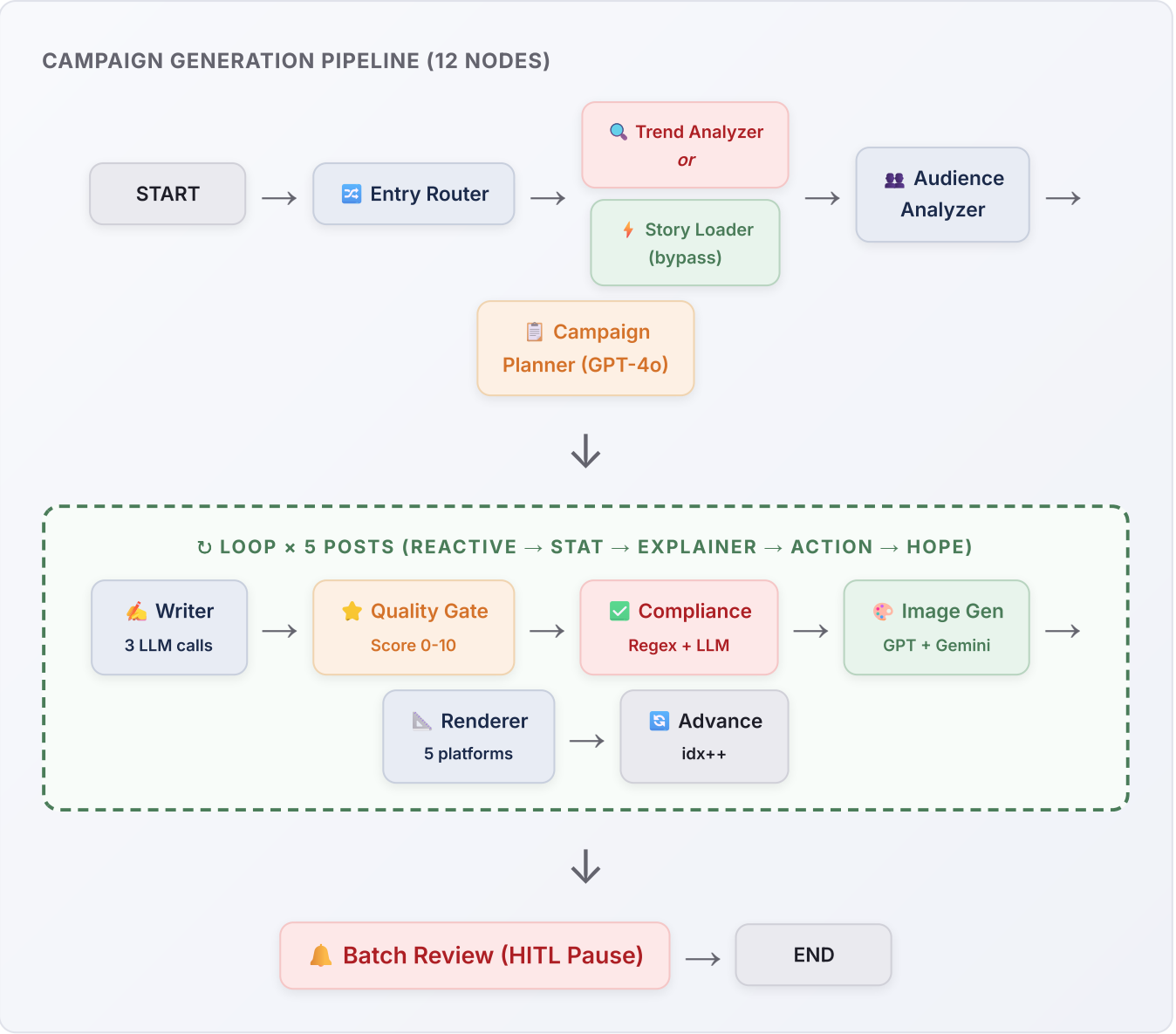
AGENT	FILE:LINE	MODEL	TEMP	STRUCTURED OUTPUT
Trend Analyzer	trend_analyzer.py:25	`gpt-4o-mini`	0	TrendAnalysis (Pydantic)
Audience Analyzer	audience_analyzer.py:47	`gpt-4o-mini`	0	AudienceDecision
Campaign Planner	campaign_planner.py:56	`gpt-4o`	0.3	CampaignBriefs (5 briefs)
Writer (primary)	writer.py:76	`gpt-4o-mini`	0.3	Raw text
Writer (variants)	writer.py:77	`gpt-4o-mini`	0.7	Raw text (higher diversity)
Quality Gate	quality_gate.py:55	`gpt-4o-mini`	0	QualityScore
Compliance Checker	compliance_checker.py:30	`gpt-4o-mini`	0	ComplianceResult
Image Gen (prompt)	image_generator.py:60	`gpt-4o-mini`	0.7	Raw text (prompt)
Image Gen (overlay)	image_generator.py:61	`gpt-4o-mini`	0	OverlayText
Publisher	publisher.py:18	`gpt-4o-mini`	0.3	Raw text
Language Adapter	language_adapter.py:35	`gpt-4o-mini`	0.2	TranslatedPost
Intel Service (ranker)	intel_service.py:35	`gpt-4o-mini`	0	StoryRankings (UNUSED)

LLM Calls Per Campaign Run (5 Posts)

AGENT	CALLS/POST	× 5 POSTS	MODEL
Trend Analyzer (or Story Loader = 0)	1	1	gpt-4o-mini
Audience Analyzer	1	1	gpt-4o-mini
Campaign Planner	1	1	**gpt-4o**
Writer (primary)	1	5	gpt-4o-mini
Writer (variant 1 + 2)	2	10	gpt-4o-mini
Quality Gate	1	5	gpt-4o-mini
Compliance Checker	1	5	gpt-4o-mini
Image Gen (prompt + overlay)	2	10	gpt-4o-mini
Image Gen (Gemini)	1-2	5-10	gemini-3.1-flash
TOTAL	**—**	**~43-48**	**3 models**

5 LangGraph Pipeline — Detailed Flow

The pipeline is implemented as a LangGraph StateGraph in `graph.py` (207 lines). It defines 12 nodes connected by conditional edges, compiled with a MemorySaver checkpointer and `interrupt_after=['batch_review']`.



> 💡 **STORY BYPASS INNOVATION**
>
> When a user picks a story from the Intel Feed, `server.py:228-232` sets `manual_url` + `story_title` in state. The `entry_router` in `graph.py:119` detects `story_title` and routes to `story_loader` (which only scrapes — **0 LLM calls**) instead of `trend_analyzer`.

Routing Logic

- **route_entry:** If `state.story_title` exists → `story_loader`. Else → `trend_analyzer`.
- **route_after_planner:** If `campaign_posts` is populated → `write_post`. Else → END.
- **route_after_render:** If `idx + 1 < len(posts)` → `advance_index` (loop). Else → `batch_review`.

6 API Endpoints (19 REST + 1 WebSocket)

#	METHOD	ENDPOINT	PURPOSE
1	GET	`/`	Serves design_v3.html dashboard
2	POST	`/api/intel/scan`	Trigger Google News RSS scan → persist stories
3	GET	`/api/intel/stories`	Get cached intelligence stories
4	POST	`/api/campaign/generate`	Start 5-post campaign from story or URL
5	GET	`/api/campaign/{id}`	Get campaign + all posts (polling endpoint)
6	POST	`/api/campaign/{id}/approve-all`	Batch approve all posts
7	POST	`/api/campaign/{id}/posts/{idx}/approve`	Approve single post
8	POST	`/api/campaign/{id}/posts/{idx}/edit`	Edit caption and/or hashtags
9	POST	`/api/campaign/{id}/posts/{idx}/regen-image`	Regenerate image with instructions
10	POST	`/api/campaign/{id}/translate`	Translate campaign to ES/FR/PT
11	GET	`/api/campaign/{id}/download`	Create + return ZIP bundle URL
12	GET	`/api/library`	All past campaigns with thumbnails
13	GET	`/api/rules`	Get content rules from YAML
14	PUT	`/api/rules`	Update content rules from dashboard
15	GET	`/api/settings`	Get application settings
16	PUT	`/api/settings`	Update settings
17	GET	`/api/audit/{id}`	Audit trail for one campaign
18	GET	`/api/audit`	Recent activity across all campaigns
19	GET	`/api/costs`	Lifetime cost summary
20	GET	`/api/costs/{id}`	Per-campaign cost breakdown by agent
21	GET	`/api/health`	Health check — returns version + feature list
22	WS	`/ws/campaign/{id}`	Real-time campaign updates (approve, edit, batch)

7 Database Schema (6 Tables)

All persistence uses SQLite via SQLModel. The database file is `redm_v3.db` (~17 MB). All 6 tables are created on server startup via `init_db()`.

ENTITY RELATIONSHIP OVERVIEW

Campaign (1)

id · source_url · source_title · status
total_cost_usd · language · thread_id · created_at

Post (5 per campaign)

campaign_id (FK) · content_type · day_of_week
caption · hashtags (JSON) · image_path · quality_score
compliance_passed · variants (JSON) · platform_renders (JSON)

IntelStory

title · source · url · snippet · full_content
relevance_score · is_ai_pick · used_in_campaign_id

AuditEntry

campaign_id · post_id · action · user_id
details (JSON) · timestamp

PromptVersion

agent · version · prompt_text
avg_quality_score · usage_count · is_active

CostLog

campaign_id · agent · model
input_tokens · output_tokens · cost_usd

8 Frontend Dashboard Features

The primary UI is `design_v3.html` (1,322 lines, 122 KB) — a single-page application with 9 views, a collapsible activity panel, a 3-step onboarding overlay, and a carousel review modal.

Dashboard Views (9)

VIEW	KEY FEATURES
Content Calendar	5-day grid with image thumbnails, quality badges, schedule selectors, audience chips, character counts, Approve/Edit/Regen per post, source story bar, weekly email preview
Intelligence Feed	Live scan button, ranked story cards with AI Pick badges, manual URL override input, "Generate Campaign" per story
Impact Tracker	KPI cards (reach, engagement, hotline clicks, cost from /api/costs), per-type performance bars, weekly trend chart
Content Library	Grid of past campaigns with thumbnail images from /api/library; click to reload into calendar
Content Rules	3-tier system: Legal (locked), Ethical Language (toggleable), Brand Voice (customizable); compliance score bar
Social Accounts	4 platform cards: IG, X, LI (connected), FB (pending)
Team Management	3 members with roles (Creator, Reviewer, Admin); invite button
Settings	Theme (light/dark), scan frequency, source priority, posts per campaign, API keys
Onboarding	3-step overlay: Welcome → Workflow → Ethics. Dismissed to localStorage

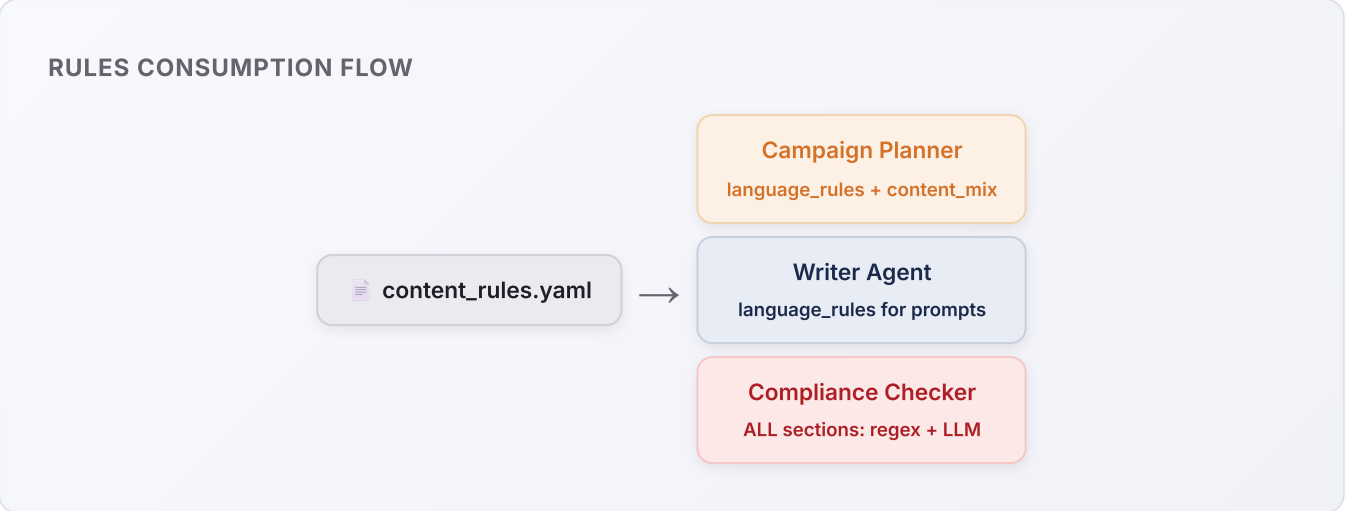
Interactive Features

FEATURE	DETAILS
Dark Mode	`data-theme="dark"` on `<html>`; persisted to localStorage
Carousel Modal	Full-screen post viewer with ←/→ nav, dot indicators, approve/edit/regen
Keyboard Shortcuts	J/K navigate, A approve, E edit, C caption, 1-5 jump to day, Esc close
Confetti	60 colored particles on batch approval via CSS @keyframes
Awareness Countdown	Sidebar countdown to next anti-trafficking day (Jul 30, Jan 11, Jan 1)
WebSocket	Connects to `ws://localhost:8000/ws/campaign/{id}` for live updates
Generation Overlay	Progress bar + step indicators during pipeline execution
Activity Panel	Collapsible sidebar with color-coded audit trail entries

FEATURE	DETAILS
Persistent Hotline	Red bar: `1-888-373-7888 / Text BeFree to 233733`
Auto-load	On load: health check → costs → audit → intel → library → latest campaign

9 Content Rules Engine

The rules system is driven by `content_rules.yaml` (65 lines), consumed by **3 agents**: Campaign Planner, Writer, and Compliance Checker.



Language Rules (6 Entries — All Enabled)

BANNED TERM	ETHICAL REPLACEMENT	REASON
"rescued"	"identified" / "supported"	Centers survivor agency
"child prostitute"	"child victim of sex trafficking"	Children cannot consent
"sex slave"	"survivor of sex trafficking"	Avoids dehumanization
"sold her body"	"was exploited"	Removes victim blame
"illegal immigrant"	"undocumented individual"	Person-first language
"the trafficking problem"	"trafficking affects [people]"	Center impacted people

Content Rules (7 Flags)

- **always_include_hotline:** true — Every post must include 1-888-373-7888 or BeFree 233733
- **must_cite_sources:** true — All statistics must have visible source attribution
- **never_identify_survivors:** true — Protect survivor privacy
- **no_sensationalized_imagery:** true — No chains, bound hands, or gratuitous imagery
- **weekly_hope_post:** true — Prevents compassion fatigue
- **avoid_racial_profiling_triggers:** true — Frame signs as observable conditions
- **Post structure:** Required sections (hook, educate, empower, source), 2-5 hashtags

Audience Profiles (6 Personas)

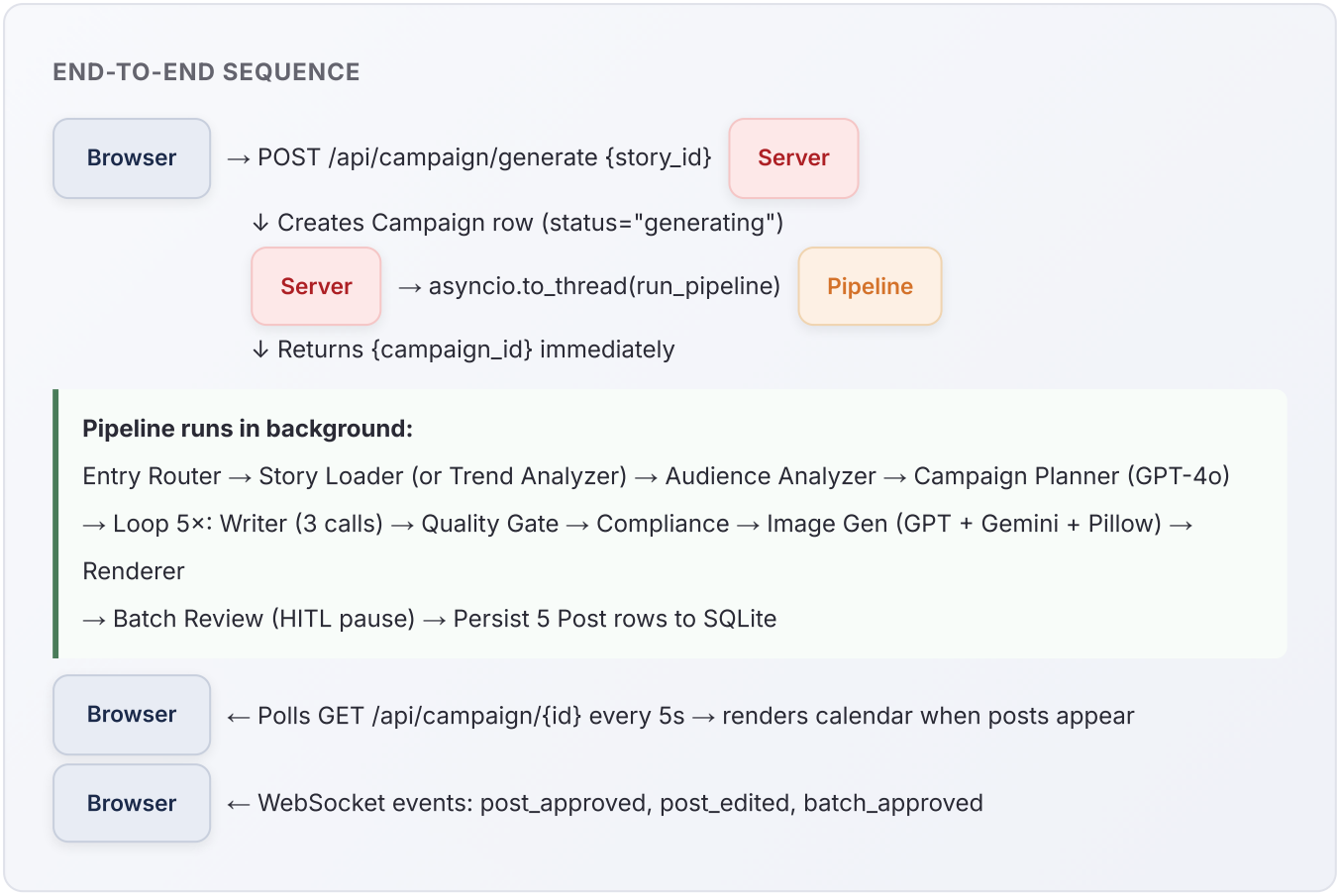
PROFILE	TONE	VISUAL STYLE	FOCUS
college_students	Direct, social-native	Dark + neon accent	Online exploitation, sextortion
educators	Professional, informative	Warm amber tones	Classroom awareness, reporting
business_owners	Compliance-focused	Cool blue-steel	Supply chain, forced labor
parents	Protective, conversational	Golden hour warmth	Online safety, grooming
lawmakers	Formal, policy-oriented	Desaturated monochrome	Legislative gaps, reform
general_public	Accessible (default)	Cinematic teal-orange	Basic awareness, hotline

10 Configuration & Dependencies

Python Dependencies (18 Packages)

CATEGORY	PACKAGE	VERSION	PURPOSE
Core AI	langchain-openai	0.3.14	OpenAI LLM wrapper
Core AI	langgraph	0.4.1	State machine pipeline
Core AI	openai	1.78.1	OpenAI SDK
Core AI	google-generativeai	0.8.5	Gemini image gen
Core AI	langsmith	0.3.42	Tracing / observability
Search	exa_py	1.9.1	Neural news search
Search	crawl4ai	0.5.0	Async web scraping
Search	tavily-python	0.5.0	⚠️ UNUSED in codebase
Web	fastapi	0.115.12	REST API framework
Web	uvicorn[standard]	0.34.2	ASGI server
Web	websockets	15.0.1	WebSocket support
Data	sqlmodel	0.0.24	ORM + Pydantic models
Data	pydantic	2.11.2	Data validation
Data	pyyaml	6.0.2	YAML config parsing
Image	pillow	≥10.4	Text overlay compositing
Schedule	apscheduler	3.10.4	Timed publishing
Memory	chromadb	0.6.3	Vector deduplication
Util	python-dotenv	1.1.0	Env file loading

11 Data Flow — End-to-End Campaign Generation



Step-by-Step

1. **Initiation:** User clicks "Generate Campaign" or pastes a URL. Frontend sends POST with `{story_id}` or `{manual_url}`.
2. **Server Processing:** Creates Campaign row (status='generating'), initializes LangGraph state, spawns background thread. Returns campaign_id immediately.
3. **Entry Routing:** If `story_title` is set → `story_loader` scrapes article (0 LLM calls). Else → `trend_analyzer` scans Google News RSS.
4. **Planning:** Audience Analyzer matches to 1 of 6 profiles. Campaign Planner (GPT-4o) creates 5 content briefs.
5. **Generation Loop (×5):** Writer generates caption + 2 variants → Quality Gate scores 0-10 → Compliance Checker validates + auto-fixes → Image Generator builds prompt + generates via Gemini + composites text overlay → Platform Renderer creates 5 platform sizes.
6. **Persistence:** Updates Campaign status to 'review', creates 5 Post rows with all data. Audit service logs the event.
7. **Human Review:** Reviewer approves/edits/regens posts. Batch approval triggers confetti. Download creates ZIP bundle.

12 Known Issues & Observations

- WARN **Font Filename Mismatch:** `image_generator.py` loads `PlayfairDisplay[wght].ttf` / `Roboto[wdth,wght].ttf` (variable font names), but actual files are `PlayfairDisplay-Bold.ttf` / `Roboto-Bold.ttf`. Falls back to Arial silently — images use wrong fonts.
- WARN **Unused Dependency:** `tavily-python==0.5.0` in requirements.txt and `TAVILY_API_KEY` in .env, but never imported anywhere. Can be safely removed.
- WARN **Unused Intel Ranker:** `IntelService.__init__()` creates a GPT-4o-mini structured output ranker, but `scan()` never calls it. All stories get `relevance_score=50.0`. Prepared but unfinished.
- NOTE **Stale Project Name:** .env sets `LANGCHAIN_PROJECT='Agent-8-Observability'` — should be `RedM-V3`.
- NOTE **Stale Document Title:** `visual_philosophy.md` opens with "Agent-9 Campaign Imagery" — old project name.
- NOTE **Empty Tests Directory:** `backend/tests/` exists but is empty. No automated test coverage.
- NOTE **Publisher Not in Pipeline:** `PublisherAgent` exists but is NOT a node in the LangGraph graph. Available for manual use only.
- NOTE **ChromaDB Not Wired:** `ContentMemory` implements dedup with ChromaDB, but neither `check_duplicate()` nor `store()` is called by any pipeline node. Built but not connected.

13 Stats Summary

Total Python source files	**22**	LLM models used	**3**
Total HTML files	**3**	LLM calls per campaign	**~43-48**
Total config files	**3 + 3 fonts**	Content types	**5**
Lines of Python code	**~2,800**	Platform renders per post	**5**
Lines of HTML/CSS/JS	**~3,055**	Audience profiles	**6**
REST API endpoints	**19 + 1 WS**	Language rules	**6 (all enabled)**
Pipeline nodes	**12**	Content rules	**7 (all active)**
AI agent classes	**8**	Supported languages	**4 (EN/ES/FR/PT)**
Service classes	**7**	Trusted news domains	**13**
Tool classes	**5**	Generated output assets	**169 files**
Database tables	**6**	Python dependencies	**18 (1 unused)**

— End of Document —

National Human Trafficking Hotline: 1-888-373-7888 | Text BeFree to 233733